SVPWM调试注入通道与高级定时器三路互补PWM，第四路触发注入通道转换心得

调试SVPWM的时候，由于复用模式没有更改GPIO引脚，导致TIM1 复用在PA上面，导致现象就是一烧录程序就无法读取到芯片，需要长按复位按键，且用调试软件把单片机存储扇区全部擦除，查看扇区也变成乱码

```
/* 上桥臂开启复用功能 */
GPIO_PinAFConfig(TIMX_PWM_CH1_GPIO_PORT,TIMX_PWM_CH1_GPIO_PINSOURCE,TIMX_PWM_CH1_AF);
GPIO_PinAFConfig(TIMX_PWM_CH2_GPIO_PORT,TIMX_PWM_CH2_GPIO_PINSOURCE,TIMX_PWM_CH2_AF);
GPIO_PinAFConfig(TIMX_PWM_CH3_GPIO_PORT,TIMX_PWM_CH3_GPIO_PINSOURCE,TIMX_PWM_CH3_AF);
/* 下桥臂开启复用功能 */
GPIO_PinAFConfig(TIMX_PWM_CH1N_GPIO_PORT,TIMX_PWM_CH1N_GPIO_PINSOURCE,TIMX_PWM_CH1N_AF);
GPIO_PinAFConfig(TIMX_PWM_CH2N_GPIO_PORT,TIMX_PWM_CH2N_GPIO_PINSOURCE,TIMX_PWM_CH2N_AF);
GPIO_PinAFConfig(TIMX_PWM_CH3N_GPIO_PORT,TIMX_PWM_CH3N_GPIO_PINSOURCE,TIMX_PWM_CH3N_AF);
```

上下桥臂未打开正确复用功能，查看数据手册，恰好复用在无法使用的引脚上，猜测这部分寄存器值是被单片机保护的，但是标准库在代码上没有验证是否可以写入，因此就直接改动这部分寄存器，导致单片机无法读取芯片，存储扇区值全部乱掉，这个引脚是13 14引脚，是给烧录口使用的，猜测原因

Port		AF0	AF1
		SYS	TIM1/2
Port A	PA0	-	TIM2_CH1_ETR
	PA1	-	TIM2_CH2
	PA2	-	TIM2_CH3
	PA3	-	TIM2_CH4
	PA4	-	-
	PA5	-	TIM2_CH1_ETR
	PA6	-	TIM1_BKIN
	PA7	-	TIM1_CH1N
	PA8	MCO1	TIM1_CH1
	PA9	-	TIM1_CH2
	PA10	-	TIM1_CH3
	PA11	-	TIM1_CH4
	PA12	-	TIM1_ETR
	PA13	JTMS-SWDIO	-
	PA14	JTCK-SWCLK	-
	PA15	JTDI	TIM 2_CH1 TIM 2_ETR

未做保护

```c
void GPIO_PinAFConfig(GPIO_TypeDef* GPIOx, uint16_t GPIO_PinSource, uint8_t GPIO_AF)
{
  uint32_t temp = 0x00;
  uint32_t temp_2 = 0x00;

  /* Check the parameters */
  assert_param(IS_GPIO_ALL_PERIPH(GPIOx));
  assert_param(IS_GPIO_PIN_SOURCE(GPIO_PinSource));
  assert_param(IS_GPIO_AF(GPIO_AF));

  temp = ((uint32_t)(GPIO_AF) << ((uint32_t)((uint32_t)GPIO_PinSource & (uint32_t)0x07) * 4)) ;
  GPIOx->AFR[GPIO_PinSource >> 0x03] &= ~((uint32_t)0xF << ((uint32_t)((uint32_t)GPIO_PinSource & (uint32_t)0x07) * 4
  temp_2 = GPIOx->AFR[GPIO_PinSource >> 0x03] | temp;
  GPIOx->AFR[GPIO_PinSource >> 0x03] = temp_2;
}
```

调试ADC通道时，此部分配置适用于规则组，未选择触发源，就是只能用软件触发

```c
ADC_StructInit(&ADC_InitStructure);
/* 关闭连续转运 */
ADC_InitStructure.ADC_ContinuousConvMode=DISABLE;
ADC_InitStructure.ADC_DataAlign=ADC_DataAlign_Right;
ADC_InitStructure.ADC_ExternalTrigConvEdge=ADC_ExternalTrigConvEdge_None;
ADC_InitStructure.ADC_ExternalTrigConv =ADC_ExternalTrigConv_T1_CC1;
ADC_InitStructure.ADC_NbrOfConversion=2;
ADC_InitStructure.ADC_Resolution=ADC_Resolution_12b;
ADC_InitStructure.ADC_ScanConvMode=ENABLE;
ADC_Init(ADC_ADCX, &ADC_InitStructure);
```

```c
/**
 * @brief  ADC规则通道参数设置
 * @param
 * @retval
 */
static void bsp_ADC_Regular_Channel_Config(void)
{

    #define     SampleTime_Regular      ADC_SampleTime_15Cycles
    ADC_RegularChannelConfig(ADC_ADCX, ADC_ADCX_CH0, 1, SampleTime_Regular);
    ADC_RegularChannelConfig(ADC_ADCX, ADC_ADCX_CH1, 2, SampleTime_Regular);
}

/**
 * @brief  ADC注入通道
```

要使得注入获取外部信号来触发，需要单独设置，并且采样时间不是越短越好，因为不同采样电阻与运放电路需要一定时间来完成自身释放，因此这个采样间隔与时间是一个值得讲究的选取与计算

```c
static void bsp_ADC_Inject_Channel_Config(void)
{
    ADC_ExternalTrigInjectedConvConfig(ADC_ADCX,ADC_ExternalTrigInjecConv_T1_CC4);
    ADC_ExternalTrigInjectedConvEdgeConfig(ADC_ADCX,ADC_ExternalTrigInjecConvEdge_Rising);

    ADC_InjectedSequencerLengthConfig(ADC1, 3);
    #define     SampleTime_Injected      ADC_SampleTime_15Cycles
    ADC_InjectedChannelConfig(ADC_ADCX, ADC_ADCX_CH2, 1, SampleTime_Injected);
    ADC_InjectedChannelConfig(ADC_ADCX, ADC_ADCX_CH3, 2, SampleTime_Injected);
    ADC_InjectedChannelConfig(ADC_ADCX, ADC_ADCX_CH4, 3, SampleTime_Injected);
    ADC_AutoInjectedConvCmd(ADC_ADCX, DISABLE);



}
```

在调试PWM CH4通道的时候，关闭CH4的PWM互补通道生成

```c
TIM_OCInitStructure.TIM_OCPolarity=TIM_OCPolarity_High ;
/* 刹车状态电平 */
TIM_OCInitStructure.TIM_OCIdleState=TIM_OCIdleState_Reset;
TIM_OCInitStructure.TIM_OCNIdleState=TIM_OCNIdleState_Reset;
TIM_OC4PreloadConfig(TIMX_SVPWM_M1, TIM_OCPreload_Enable);
TIM_OC4Init(TIMX_SVPWM_M1, &TIM_OCInitStructure);

/* 断路时间配置 */
```

并且这个CH4的CCR4选取很有深度，因为这个PWM模式是中央对齐，且PWM模式1，CNT小于CCR4的时候是高电平有效，但是此时不会产生CC4事件来触发ADC注入通道，要使得比较事件后，改变极性那么就是低电平，又由于是中央对齐模式，那么会改变回高电平，此时产生CC4上升沿的比较事件，那么就会产生ADC采样注入通道，且，此时刚好是三路PWM上半PWM全是低电平时刻，那么下半桥就为高电平，此时采样最为准确，因为中央对齐模式PWM其实是高低高如此排布，但是在整个运行周期可以看成低高低模式。

```
    TIMX_SVPWM_M1->CCR3=(TIMX_SVPWM_PERIOD>>1);
    /* 保证PWM周期时，低于CCR比较值，高电平，触发ADC*/
    TIMX_SVPWM_M1->CCR4=(TIMX_SVPWM_PERIOD-1);
```

如图分析，绿色ADC中断翻转电平，每次翻转时刻都是在上半桥关闭时刻，因此在整个周期上来看 可以看成PWM刚开始的时候，其实时实际产生波形高低高 仔细分析-。-，需要完全对应上，就需要改中央对齐模式或者PWM模式

互补通道的设置，开启互补通道，会自动取反互补通道的PWM模式，因此有效电平都采取高电平

```
/* 设置OC模式 */
TIM_OCStructInit(&TIM_OCInitStructure);
TIM_OCInitStructure.TIM_OCMode=TIM_OCMode_PWM1;
/* 互补通道输出使能 */
TIM_OCInitStructure.TIM_OutputState=TIM_OutputState_Enable;
TIM_OCInitStructure.TIM_OutputNState=TIM_OutputNState_Enable;
TIM_OCInitStructure.TIM_OCPolarity= TIM_OCPolarity_High ;
/* 开启互补功能自动取反PWM模式,猜测已经验证————————————————————————
TIM_OCInitStructure.TIM_OCNPolarity=TIM_OCNPolarity_High;
/* 刹车状态电平 */
TIM_OCInitStructure.TIM_OCIdleState=TIM_OCIdleState_Reset;
TIM_OCInitStructure.TIM_OCNIdleState=TIM_OCNIdleState_Reset;
```

死区时间计算，MOS管的死区时间合理为 （110-96）+（43-16） 乘上1.2 约50ns 需要50ns，

$t_{d(on)}$	Turn-On Delay Time	——	16	——	ns	V_{DD} = 49V
t_r	Rise Time	——	110	——		I_D = 46A
$t_{d(off)}$	Turn-Off Delay Time	——	43	——		R_G = 6.8Ω
t_f	Fall Time	——	96	——		V_{GS} = 10V ⑤

一般都是0分频 也就是168Mhz

TIMx_CR1

CKD[1:0]：时钟分频 (Clock division)

此位域指示定时器时钟 (CK_INT) 频率与死区发生器以及数字滤波器（ETR、TIx）所使用的死区及采样时钟 (t_{DTS}) 之间的分频比，

00：$t_{DTS}=t_{CK_INT}$
01：$t_{DTS}=2*t_{CK_INT}$
10：$t_{DTS}=4*t_{CK_INT}$
11：保留，不要设置成此值

TIMx_BDTR

DTG[7:0]：配置死区发生器 (Dead-time generator setup)

此位域定义插入到互补输出之间的死区持续时间。DT 与该持续时间相对应。

DTG[7:5]=0xx => DT=DTG[7:0]x t_{dtg}，其中 $t_{dtg}=t_{DTS}$。
DTG[7:5]=10x => DT=(64+DTG[5:0])xt_{dtg}，其中 $T_{dtg}=2xt_{DTS}$。
DTG[7:5]=110 => DT=(32+DTG[4:0])xt_{dtg}，其中 $T_{dtg}=8xt_{DTS}$。
DTG[7:5]=111 => DT=(32+DTG[4:0])xt_{dtg}，其中 $T_{dtg}=16xt_{DTS}$。
示例：如果 T_{DTS}=125ns (8MHz)，则可能的死区值为：
0 到 15875 ns（步长为 125 ns）
16 us 到 31750 ns（步长为 250 ns）
32 us 到 63us（步长为 1 us）
64 us 到 126 us（步长为 2 us）

死区时间计算分三步走：

第一步：通过 CKD[1:0]位确定 tDTS。根据 CKD[1:0]位的描述，可以得到下面的式子：

$$\text{tDTS} = \frac{2\text{^}CKD[1:0]}{\text{Tclk}}$$

CKD[1:0]：CKD[1:0]位设置的值。

Tclk：定时器的时钟源频率（单位为 MHz）。

假设定时器时钟源频率是 168MHz，我们设置 CKD[1:0]位的值为 2，代入上面的式子可得：

$$\text{tDTS} = \frac{2\text{^}CKD[1:0]}{\text{Tclk}} = \frac{2\text{^}2}{168000000} = \frac{4}{168000000} = 23.81\text{ns}$$

第二步：代入选择的公式计算。

下面给大家举个例子，假设定时器时钟源频率是 168MHz，我们设置 CKD[1:0]位的值为 2，DTG[7:0]位的值为 250。从上面的例子知道 CKD[1:0]位的值为 2，得到的 tDTS=23.81ns。下面来看一下 DTG[7:0]位的值为 250，应该选择 DTG[7:0]位描述中哪条公式？250 的二进制数为 11111010，即 DTG[7:5]为 111，所以选择第四条公式：DT=(32+ DTG[4:0]) * t_{dtg}，其中 t_{dtg} = 16 * tDTS。可以看到手册上的式子符号大小写乱乱的，这里大小写不敏感。由手册的公式可以得到 DT = (32+ DTG[4:0]) * 16 * tDTS = (32+ 26) * 16 * 23.81ns = 22095.68ns = 22.01us，即死区时间为 22.01us。死区时间计算方法就给大家介绍到这里。

T DTS 也就是时间系数 2次方CKD[1:0] 一般都是0，也就是1/168Mhz=5.7ns

死区时间填0xFF 高三位111那么选第四条公式（32+31）* 16 * 5.7 = 5.8us

实测波形5.88us左右 因此推算成立